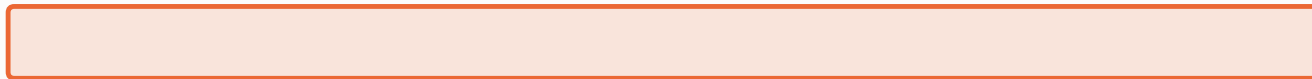


Why MLA: one shared latent instead of 128 K/V pairs

KV cache per token

standard MHA — 128 heads x (192 K + 128 V) = 40,960 values



→ MLA — $c_{kv} 512 + pe 64 = 576$ values (sliver drawn to the SAME scale: 1/71 of the bar above)

The absorbed path: move the per-head matrix onto the query side

$$q_{nope}[h] \cdot K[h] = q_{nope}[h] \cdot (c_{kv} W_{uk}[h]^T) = (q_{nope}[h] W_{uk}[h]) \cdot c_{kv}$$

reconstruct K for 128 heads
 $5k \times 128 \times 192$ values

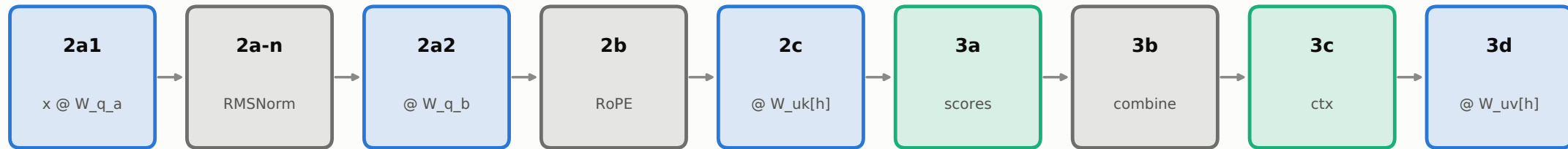
absorb into the query, ONCE
 $1 \times 128 \times 512$ values

c_{kv} and pe_cache carry no head index: all 128 heads of a query read the SAME cache rows. That is why kernel 3a gives one block all 128 heads at once — it stages the cache once and reuses it 128 times.

The 9 steps

Kernel 2 builds the 576-wide query; kernel 3 does the attention. Shapes for batch-128 decode.

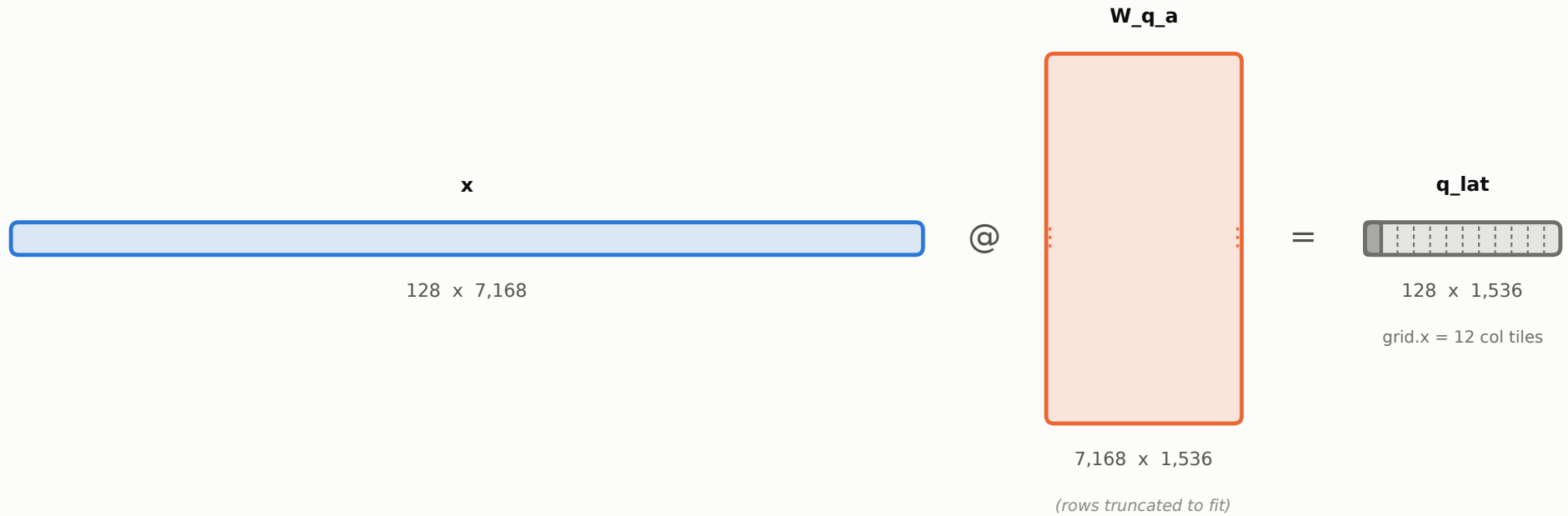
KERNEL 2 — Q path



Step 2a1 — project the hidden state into the q_lora latent

$$q_lat = x @ W_q_a$$

the first half of the factorised Q projection



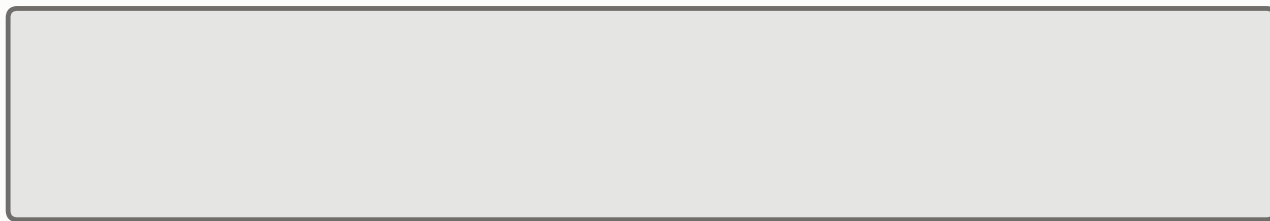
grid.x -> 12 output-column tiles of 128 | grid.y -> 1 row tile (bq = 128) | grid.z -> split-K: the 7,168 reduction is cut into slices
Only 12 column tiles means 12 blocks on an 84-SM GPU — split-K is what makes this step use the machine at all.

Step 2a-norm — RMSNorm on the latent

$q_lat := q_lat * \text{rsqrt}(\text{mean}(q_lat^2) + \text{eps}) * \text{gain}$

row-wise; keeps the two matmuls from collapsing into one rank-1536 matrix

q_lat (in and out)



128 x 1,536

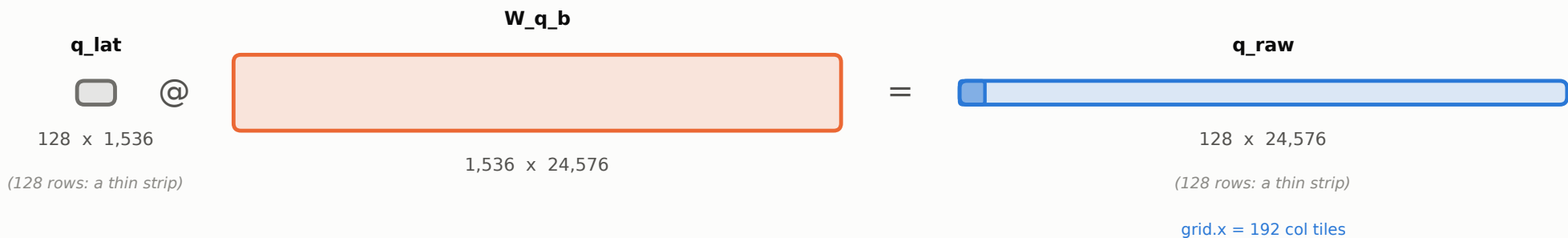
one CUDA block per row -> 128 blocks
256 threads cover 1,536 elements (6 each)
block-wide sum by warp shuffles, then one
pass over the per-warp partials

Reads 2a1's wide fp32 accumulator and writes the narrow input dtype, which is what lets step 2a2 reuse the same GEMM kernel unchanged.

Step 2a2 — expand the latent into the per-head query

$$q_{\text{raw}} = q_{\text{lat}} @ W_{q_b} \quad (24,576 = 128 \text{ heads} \times 192)$$

the second half of the factorised Q projection — the largest weight in the model at 151 MB



grid.x -> 192 output-column tiles of 128 | grid.y -> 1 row tile | grid.z -> split-K over the 1,536 reduction
192 blocks against 84 SMs x 2 resident = 168 slots is 1.14 waves: one full wave, then a tail wave leaving 144 slots idle.

Step 2b — RoPE on the rope half of each head

`q_rope[b,s,h,:] = rotate( q_raw[b,s,h, 128:192], position[s] )`

elementwise; rotates adjacent pairs by an angle set by the query's ABSOLUTE position

q_raw, ONE head: 192 values



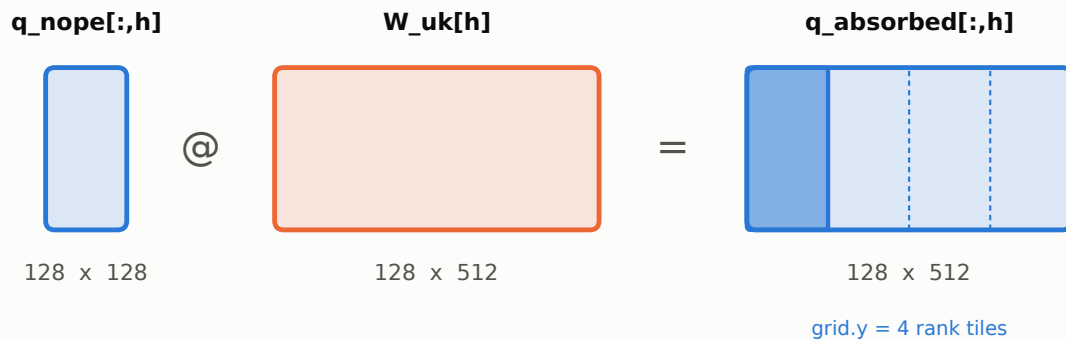
over all rows and heads: `[128 x 128 x 192] -> q_rope [128, 128, 64]`

One thread per (row, head, adjacent pair), grid-stride capped at 32 x SM so the launch never scales with the cache depth. 0.1% of decode.

Step 2c — absorb W_{uk} into the query (once per head)

$q_absorbed[:,h] = q_raw[:,h, 0:128] @ W_uk[h]$ repeated for $h = 0 \dots 127$

this is the absorption: it widens the query 128 -> 512 so the cache can be read in its stored latent form

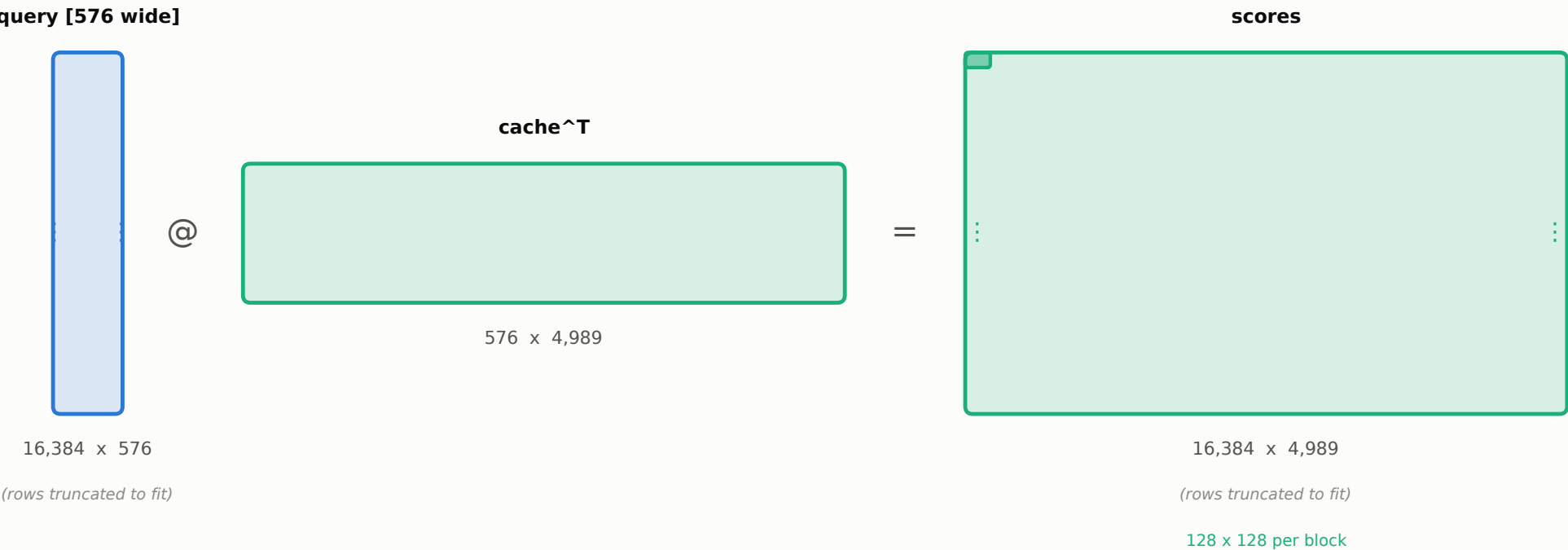


grid.x -> bq row tiles | grid.y -> 4 tiles of 128 over kv_rank=512 | grid.z -> the head index, 0..127
One head per block means $W_{uk}[h]$ (262 KB) is staged into shared memory once and reused by every row in the tile.

Step 3a — scores, then the local softmax numerator

$\text{scores} = (q_{\text{absorbed}} \cdot c_{\text{kv}} + q_{\text{rope}} \cdot \text{pe_cache}) / \text{sqrt}(192)$ then store $\exp(s - m_j)$

the two dot products (512 and 64 deep) are run as ONE 576-deep reduction; shown at $S_k = 4,989$

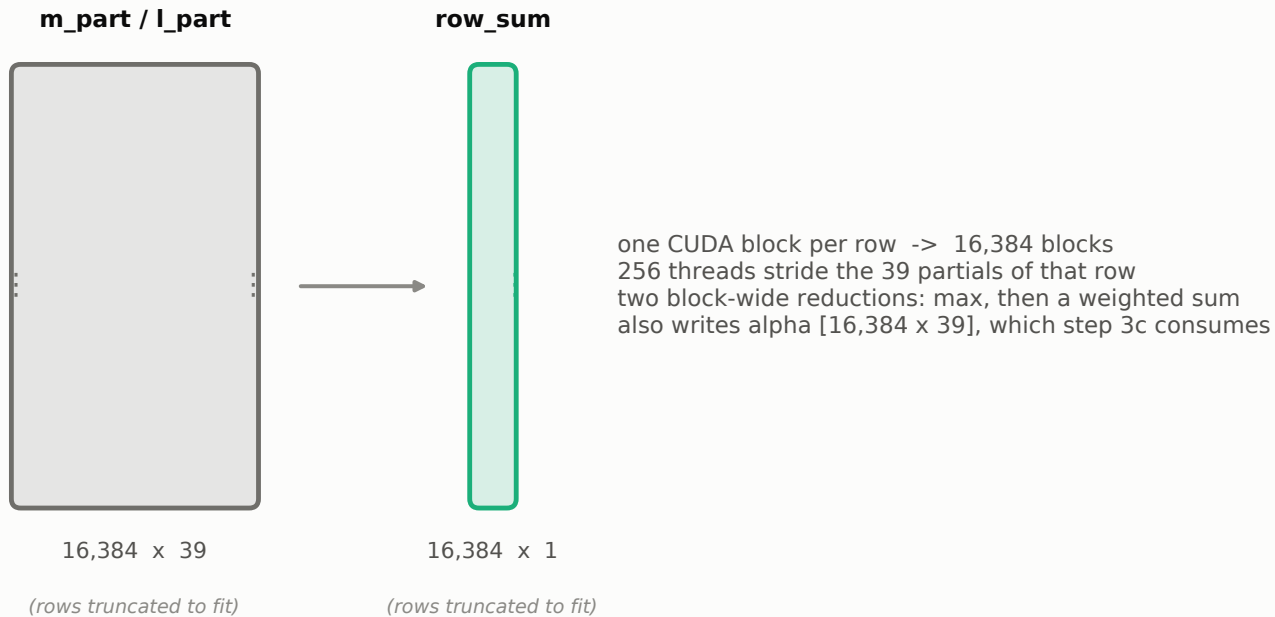


grid.x -> flat = $B.Sq.H / 128 = 128$ tiles | grid.y -> $S_k / 128 = 39$ tiles | no grid.z (scores has a real S_k axis)
A block's 128 rows are the 128 heads of ONE query, so the cache tile it stages is reused by all 128 of them.

Step 3b — reconcile the per-block softmax maxima

$$M = \max_j m_j \quad \alpha_j = \exp(m_j - M) \quad \text{row_sum} = \sum_j l_j * \alpha_j$$

3a's grid.y split each row across 39 blocks, none of which could see the global maximum

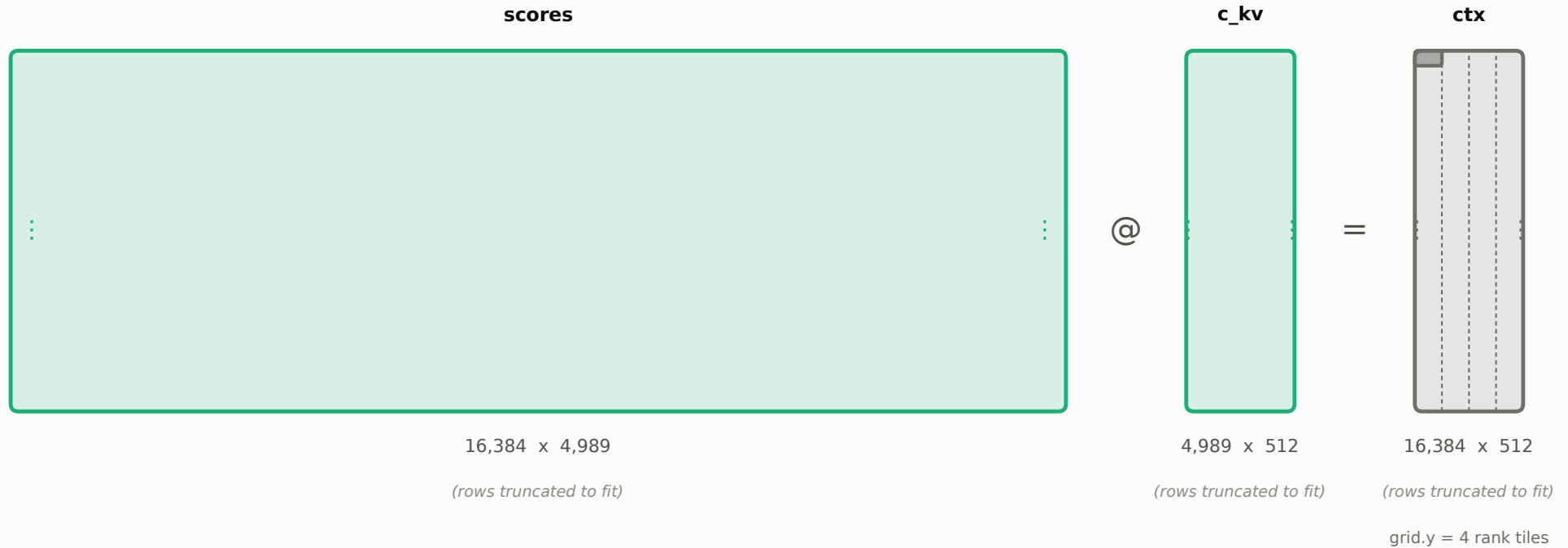


Exact because $\exp(s - m_j) * \exp(m_j - M) = \exp(s - M)$ termwise. Total storage 7.7 MB against a 327 MB scores tensor; costs 0.5% of decode.

Step 3c — context: the attention-weighted sum of the cache

$$\text{ctx} = (\exp(s - m_j) * \alpha_j) @ c_{kv} = \exp(s - M) @ c_{kv}, \text{ unnormalised}$$

α_j is folded into the staging load, so what lands in shared memory is already the softmax numerator

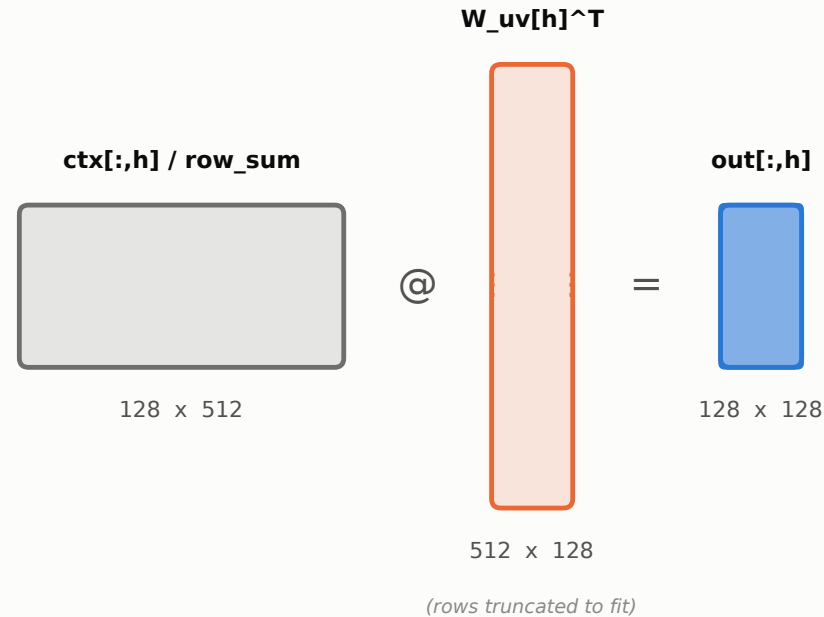


grid.x -> flat / 128 = 128 tiles | grid.y -> 4 tiles over kv_rank = 512 | grid.z -> split-K over Sk
ctx has no Sk axis, so at B=Sq=1 the output alone yields 4 blocks and ~95% of the SMs would idle — hence split-K here.

Step 3d — reconstruct V and project out (once per head)

$\text{out[:,h]} = (\text{ctx[:,h]} / \text{row_sum}) @ W_{uv}[h]^T$ repeated for $h = 0 \dots 127$

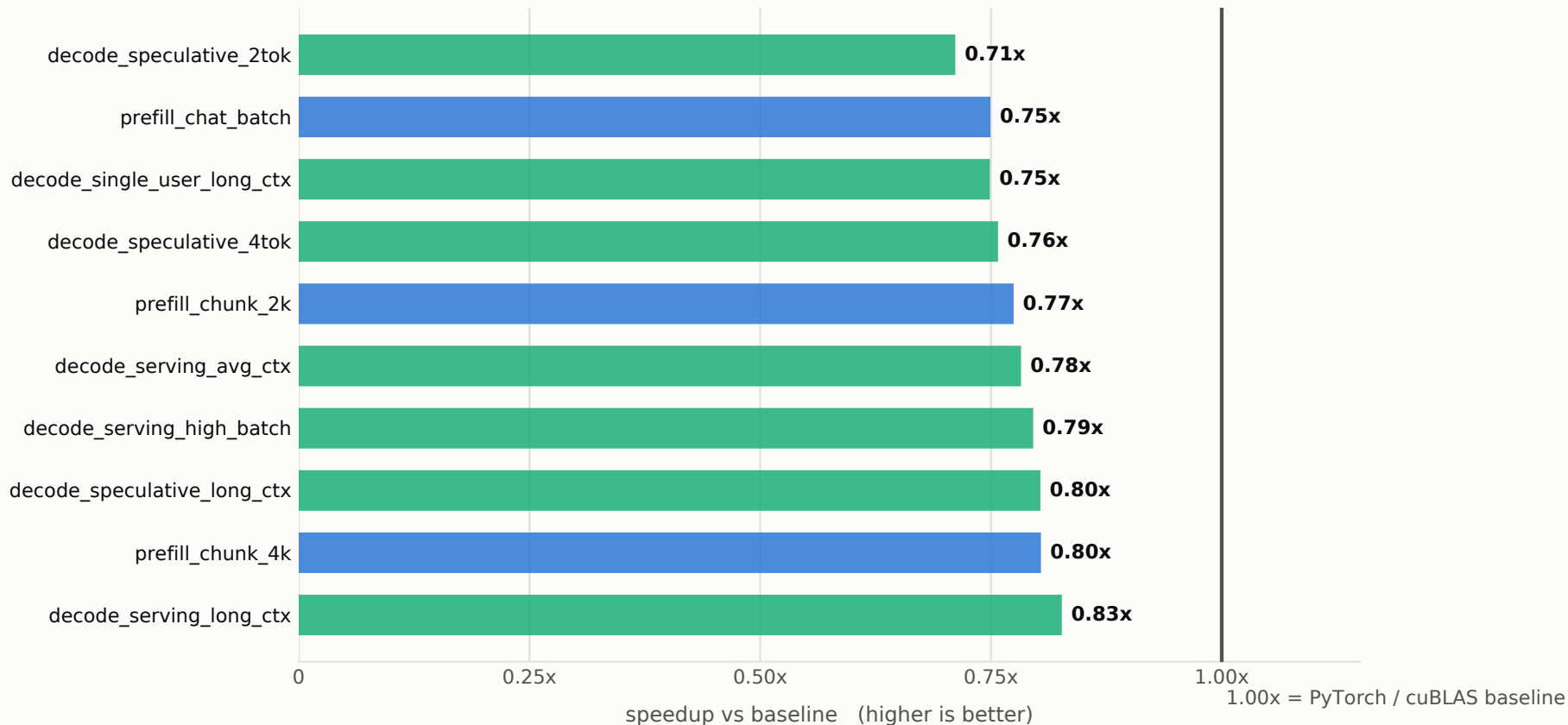
the deferred softmax divide rides along on a staging load this kernel was already doing



grid.x -> bq row tiles | grid.y -> 1 tile over $v_dim = 128$ | grid.z -> the head index, $0..127$
Narrows 512 -> 128, undoing the widening that step 2c did. One head per block, same reuse argument as 2c.

End-to-end result: 0.77x geometric mean

FP32, RTX A6000, L2 flushed between iterations, TF32 disabled on both sides. All 10 scenarios pass correctness.



Decode (aqua) and prefill (blue) land in the same narrow band: the tiling generalises, but cuBLAS keeps a ~23% edge.